

Names: Ishimwe Pacis Hanyurwimfura
Student Number: 3787234

[Link to Excel with results running time for different modes and different Matrix sizes \(512, 1024, 2048 \)](#)

[Link to Github](#)

Assignment 1 Discussion

1. Static vs dynamic scheduling: which was better and why?

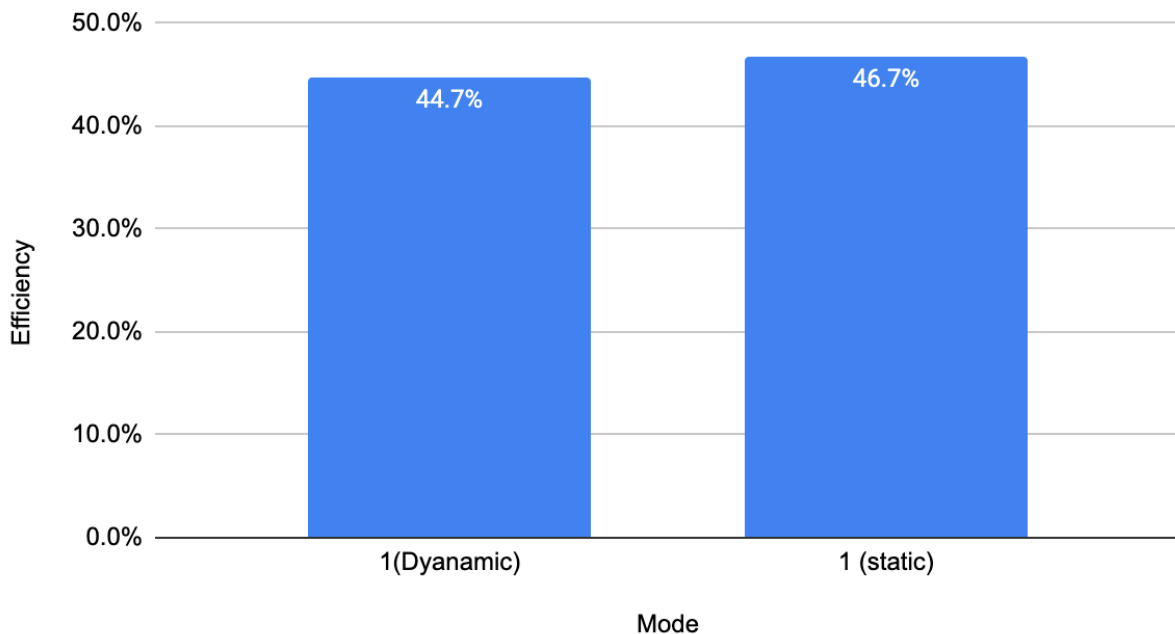
ANS:

Overall static scheduling performed well as:

- The matrix multiplication had a uniform workload per iteration
- There was no load imbalance
- Dynamic would have worked better if workload per iteration was varied or changing

Diagram Comparison of Mod1 N 2048 static vs dynamic

Efficiency Static Vs Dynamic Comparison- Mode 1 N 2048



2. Why does efficiency drop as thread count increases?

ANS:

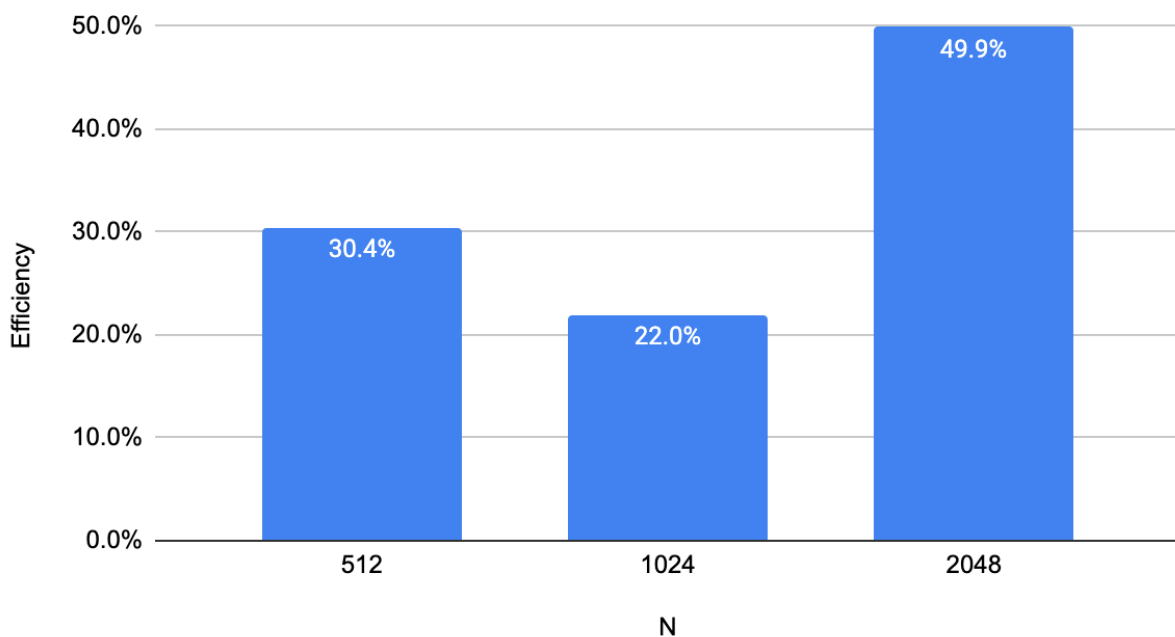
Efficiency drops as threads increase because:

- As threads increase they compete for resources (memory bandwidth)
- Also according to Amhadah's law the serial portion limit's the parallel part
forexample incases we uses atomic or critical

But also Looking at our results we can say that small problem sizes suffer more from overhead as when it went it was Large Size(2048) the speedup and Efficiency went up

Diagram

Mode 2 (16threads) - Efficiency Vs Problem Size



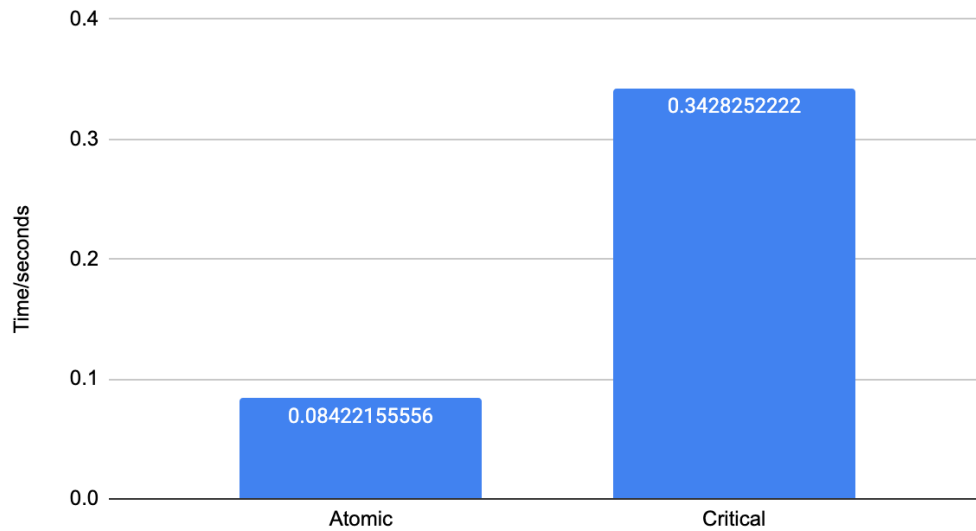
3. Atomic vs critical: which was faster for checksum, and why?

ANS:

Atomic was faster than critical as it locks memory location of checksum while critical locks a whole block and causes the threads to wait one by one, And as checksum is a small update with high frequency with critical it causes bottlenecks.

Diagram

Atomic Vs Critical - Size(2048)



4. Tasks vs loops: when did tasks help/hurt?

ANS:

In this task we can see that tasks didn't make a huge difference to loops as we were working with regular workload and with task there was task creation cost, scheduling cost and synchronization which limited it, Tasks would be way beneficial in irregular workload.

Comparison Result table of Loop (Mode 2) Vs Task (Mode 4)

Mode	N	Speedup	Efficiency
2	512	4.9	30.4%
4	512	5.34	33.4%
2	1024	3.5	22.0%
4	1024	3.50	21.9%
2	2048	8.0	49.9%
4	2048	7.49	46.8%

5. SIMD: which part benefited most, and did SIMD reduce the need for many threads?

ANS:

SIMD benefited from the K loop as it was arithmetic operation, totally independent and great for vectorization but again it didn't remove the need of threads as we can see in the

results, For large size mode 5 the average time was 46.1 seconds while parallel was 6 seconds but comparing we could see a small increase comparing serial (mode 0) vs SMD (mode 5) and also loop parallel vs threads and SIMD (mode6)

SUMMARY TABLE WITH RUN RESULTS (EFFICIENCY & SPEEDUP)

Mode	N	Speedup	Efficiency
1	512	5.39	33.7%
2	512	4.9	30.4%
4	512	5.34	33.4%
6	512	5.51	34.46%
1	1024	3.56	22.3%
2	1024	3.5	22.0%
4	1024	3.50	21.9%
6	1024	3.38	21.12%
1	2048	7.15	44.7%
2	2048	8.0	49.9%
4	2048	7.49	46.8%
6	2048	7.15	44.69%

To Note: The full details of how it was calculated and all the runs can be found in the excel [link](#) at the top or the csv file shared.